Item	Details
Scenario	SOC Alert-Triage Module of "SentinelOps": alert sorting, blocklist check, signature search, closest-pair of logins
Course outcome assessed	Apply the Brute Force technique to sorting, searching, Closest-Pair and Convex-Hull problems
Maximum marks / mode	50 marks | Individual
Student Reg. No.	192511149
Student Name	S.DEVI SHREE

Part	Task
(a)	Reproduce and identify the defects
(b)	Explain the root causes and debug systematically
(c)	Optimise the corrected module
(d)	Analyse before and after
(e)	Deliver production-quality code

Conventions used throughout

• Line numbers refer to Listing 1 (the original code). Defects are tagged **D1 … D9**. Alerts are written (id, severity).

• Every output, count and timing in this document was produced by running the code (files: sentinelops_original.py, sentinelops.py, test_sentinelops.py, experiments.py) on Python 3; nothing is estimated except where the text says "extrapolated".

• The indentation of the code in the question PDF was checked on the page image: the `else:` on line 17 is level with the `if` on line 15, so it belongs to the `if` inside the loop.

Part (a) – Reproduce and Identify the Defects

A.1 The original code, line-numbered

The module was saved unchanged as sentinelops_original.py and run on the test data. Listing 1 shows it with line numbers; the nine defective lines are shaded and tagged (the tags are my annotations).

```
 1  import math
 2
 3  def sort_alerts(alerts):
 4      for i in range(1, len(alerts)):
 5          key = alerts[i]
 6          j = i - 1
 7          while j > 0 and alerts[j][1] < key[1]:        # <-- D1
 8              alerts[j + 1] = alerts[j]
 9              j -= 1
10          alerts[j] = key        # <-- D2
11      return alerts
12
13  def is_blocked(blocklist, ip):
14      for i in range(len(blocklist)):
15          if blocklist[i] == ip:
16              return False       # <-- D3
17          else:       # <-- D4
18              return False       # <-- D4
19
20  def find_signature(log, pattern):
21      n, m = len(log), len(pattern)
22      hits = []
23      for i in range(n - m):        # <-- D5
24          for j in range(m):
25              if log[i + j] != pattern[j]:
26                  break
27          if j == m - 1:        # <-- D6
28              hits.append(i)
29      return hits
30
31  def closest_logins(events):
32      min_dist = 0        # <-- D7
33      pair = None
34      for i in range(len(events)):
35          for j in range(len(events)):        # <-- D9
36              d = math.sqrt((events[i][0] - events[j][0]) ** 2 +
37                            (events[i][1] - events[j][1]) ** 2)
38              if d > min_dist:        # <-- D8
39                  min_dist = d
40                  pair = (i, j)
41      return pair, min_dist
42
43  if __name__ == "__main__":
44      alerts    = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]
45      blocklist = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]
46      queries   = ["10.0.0.9", "10.0.0.5", "8.8.8.8"]
47      log, pattern = "FAILFAIXFAIL", "FAIL"
48      events    = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]
49      print(sort_alerts(list(alerts)))
50      print([is_blocked(blocklist, q) for q in queries])
51      print(find_signature(log, pattern))
52      print(closest_logins(events))
```

Listing 1 – original code with defect tags (shaded).

A.2 Expected versus actual output for every function

How the expected outputs were obtained. They were derived by hand from the stated purpose of each function, before running anything: (1) severities 9, 9, 5, 3, 1 in descending order, with A2 ahead of A4 because it arrived first; (2) 10.0.0.9 and 10.0.0.5 are on the blocklist, 8.8.8.8 is not; (3)

FAIL starts at index 0 and index 8 (index 4 reads FAIX, which is not FAIL); (4) of the 15 pairs of locations the closest are E0 = (2, 3) and E5 = (3, 4), at distance $\sqrt{2} \approx 1.414$ km (full table in Part (b)).

Function	Input	Expected output	Actual output	Verdict
sort_alerts	alerts (5 tuples)	[(A2, 9), (A4, 9), (A3, 5), (A1, 3), (A5, 1)]	[(A4, 9), (A3, 5), (A3, 5), (A5, 1), (A5, 1)]	FAIL – A1, A2 lost; A3, A5 duplicated
is_blocked	"10.0.0.9"	True (2nd entry)	False	FAIL
is_blocked	"10.0.0.5"	True (1st entry)	False	FAIL
is_blocked	"8.8.8.8"	False	False	pass – by accident (never returns True)
find_signature	"FAILFAIXFAIL", "FAIL"	[0, 8]	[0, 4]	FAIL – false hit at 4, hit at 8 missed
closest_logins	events (6 points)	((0, 5), 1.4142)	((0, 2), 60.4401)	FAIL – returns the FARTHEST pair

A.3 Defect register – function, faulty line, nature of the error

ID	Function	Line	Faulty code	Nature of the error	Observed effect
D1	sort_alerts	7	while j > 0 and …	Boundary / off-by-one in the loop guard	Index 0 is never compared or shifted, so nothing can be inserted in front of it
D2	sort_alerts	10	alerts[j] = key	Wrong index when storing (off by one)	Overwrites a live alert: one alert lost, a shifted copy duplicated
D3	is_blocked	16	return False (in the if)	Wrong return value (inverted logic)	A matching IP is reported as "not blocked"
D4	is_blocked	17–18	else: return False	Premature return inside the loop (misplaced else)	Function quits after the first entry; entries 2…b are never examined
D5	find_signature	23	range(n - m)	Off-by-one loop bound	Last alignment i = n − m is never tried: pattern at the end is missed
D6	find_signature	27	if j == m - 1:	Faulty match test (control flow)	True for a full match and for a mismatch at the last character: false signatures
D7	closest_logins	32	min_dist = 0	Wrong initial value (needs +∞)	No distance can be below 0, so a "minimum" search never updates
D8	closest_logins	38	if d > min_dist:	Wrong comparison operator	Keeps the maximum: the farthest pair is returned
D9	closest_logins	35	for j in range(len(events))	Wrong loop range	Self pairs (d = 0) and mirrored pairs (i, j)/(j, i) are evaluated

Count: 2 + 2 + 2 + 3 = **9 defects**, matching the statement of the task.

A.4 Traceability – production symptom → defect

Production symptom	Defects	Evidence on the test data
Alerts disappeared or were duplicated	D2 (with D1)	A1 and A2 vanish; A3 and A5 appear twice
Blocked IP addresses got through	D3, D4	Both listed IPs return False
Signature at the end of a log was missed	D5	Window i = 8 ("FAIL") is never examined
False signatures were reported	D6	Index 4 ("FAIX") is reported as a hit
"Closest logins" hundreds of km apart	D7, D8, D9	Pair (0, 2), 60.44 km, is returned: the farthest pair

Part (b) – Root Causes and Systematic Debugging

Method. For each function: (1) form a hypothesis from the symptom; (2) confirm it by tracing the code by hand on the test data; (3) make the smallest possible edit – the functions are corrected, not rewritten; (4) re-run the test data and the boundary tests after every single edit. Functions are corrected one at a time, defects in the order a developer meets them.

B.1 sort_alerts (D1, D2)

Root causes and how they produce the wrong output

- **D1 – `while j > 0`.** After the key is taken, j starts at i − 1 and must be able to reach index 0, otherwise alerts[0] is never compared with the key. The guard should be `j >= 0`. With `j > 0` the scan stops at j = 1, so an alert more severe than alerts[0] can never move in front of it.
- **D2 – `alerts[j] = key`.** When the while loop ends, j points to the last element that is **not** smaller than the key (or to −1 if every element was smaller); the key belongs one place to its right, at j + 1. Writing it at j overwrites an alert that should stay: that alert is lost, and its shifted copy at j + 1 remains, so it appears twice. (When j = −1, Python's negative indexing even writes to the **last** element.)

Trace of the defective code (alerts from the test data)

Pass i	key	Loop stops at j	Shifts	List after `alerts[j] = key`
1	A2:9	0	0	[A2:9, A2:9, A3:5, A4:9, A5:1]
2	A3:5	1	0	[A2:9, A3:5, A3:5, A4:9, A5:1]
3	A4:9	0	2	[A4:9, A3:5, A3:5, A3:5, A5:1]
4	A5:1	3	0	[A4:9, A3:5, A3:5, A5:1, A5:1]

Pass 1: key A2:9, j = 0, so the loop is skipped and `alerts[0] = key` **overwrites A1** → A1 is gone and A2 is duplicated. Pass 3: key A4:9 shifts twice and is written to index 0, overwriting the copy of A2. The final list has only 3 distinct alerts out of 5 – exactly the "disappeared or duplicated" symptom.

Correcting one defect at a time, re-testing after each edit

Stage	Edit	Output on test data	Verdict
0	original	[(A4, 9), (A3, 5), (A3, 5), (A5, 1), (A5, 1)]	alerts lost / duplicated
1	D2: `alerts[j]` → `alerts[j + 1]`	[(A1, 3), (A2, 9), (A4, 9), (A3, 5), (A5, 1)]	nothing lost, but A1 stuck in front of A2 (D1 remains)
2	D1: `j > 0` → `j >= 0`	[(A2, 9), (A4, 9), (A3, 5), (A1, 3), (A5, 1)]	correct and stable
(ref)	D1 alone, without D2	[(A1, 3), (A1, 3), (A1, 3), (A1, 3), (A4, 9)]	worse: j = −1 writes to alerts[−1]

Fixing D2 first is the natural order (the symptom is data loss). The reference row shows why both edits are needed: D1 alone lets j reach −1, and `alerts[-1]` then silently overwrites the last element.

Trace of the corrected insertion sort

Pass i	key	Comparisons (`alerts[j].sev < key.sev`)	#cmp	Inserted at	List after the pass
1	A2:9	3<9?Y	1	0	[A2:9, A1:3, A3:5, A4:9, A5:1]
2	A3:5	3<5?Y 9<5?N	2	1	[A2:9, A3:5, A1:3, A4:9, A5:1]
3	A4:9	3<9?Y 5<9?Y 9<9?N	3	1	[A2:9, A4:9, A3:5, A1:3, A5:1]
4	A5:1	3<1?N	1	4	[A2:9, A4:9, A3:5, A1:3, A5:1]

Totals: 1 + 2 + 3 + 1 = **7 comparisons** and 1 + 1 + 2 + 0 = **4 shifts**. The 4 shifts equal the 4 inversions of the input (A1<A2, A1<A3, A1<A4, A3<A4), as theory predicts. **Stability:** the test is the strict

`<`, so on equal severity the loop stops and the key stays to the right of its equal: in pass 3, `9<9? N` stops A4 right behind A2, keeping arrival order.

Corrected function

```
def sort_alerts(alerts):            # insertion sort by severity, highest first
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1
        while j >= 0 and alerts[j][1] < key[1]:    # D1: was  j > 0
            alerts[j + 1] = alerts[j]
            j -= 1
        alerts[j + 1] = key                        # D2: was  alerts[j]
    return alerts
```

Boundary tests – sort_alerts

Case	Input	Original	Corrected	Expected	OK
empty list	[]	[]	[]	[]	✓
one alert	[(X, 4)]	[(X, 4)]	[(X, 4)]	[(X, 4)]	✓
equal severities	[(a, 5), (b, 5), (c, 5), (d, 5)]	[(b, 5), (c, 5), (d, 5), (d, 5)]	[(a, 5), (b, 5), (c, 5), (d, 5)]	[(a, 5), (b, 5), (c, 5), (d, 5)]	✓
ascending (worst)	[(a, 1), (b, 2), (c, 3), (d, 4)]	[(d, 4), (b, 2), (b, 2), (b, 2)]	[(d, 4), (c, 3), (b, 2), (a, 1)]	[(d, 4), (c, 3), (b, 2), (a, 1)]	✓
two alerts	[(low, 1), (high, 9)]	[(high, 9), (high, 9)]	[(high, 9), (low, 1)]	[(high, 9), (low, 1)]	✓

Equal severities keep arrival order a, b, c, d (stability). The ascending input (worst case: every alert is more severe than all before it) is fully reversed. Cases with two or more alerts that need to reach index 0 (“two alerts”, “ascending”) were already wrong in the original only when the key had to pass alerts[0] – see the original column.

B.2 is_blocked (D3, D4)

Root causes

- **D3 – `return False` after a match (line 16).** The function exists to answer “is this IP on the list?”; finding it must return **True**. The inverted value makes every blocked address look clean.
- **D4 – `else: return False` inside the loop (lines 17–18).** The `else` belongs to the `if`, so it runs for **every non-matching entry**, including the very first. The function therefore decides after looking at one entry only; entries 2 … b are unreachable. The correct “not found” answer may be returned only after the whole loop has finished.

Trace of the defective code

Query	Iteration i = 0 compares	Branch taken	Returned	Should be
“10.0.0.9”	“10.0.0.5” = “10.0.0.9”? No	else → return False (never reaches i = 1, where the match is)	False	True
“10.0.0.5”	“10.0.0.5” = “10.0.0.5”? Yes	if → return False (D3)	False	True
“8.8.8.8”	“10.0.0.5” = “8.8.8.8”? No	else → return False	False	False

One defect at a time

Stage	Edit	Results for the 3 queries	Verdict
0	original	[False, False, False]	all False
1	D3: `return False` → `return True` (match)	[False, True, False]	10.0.0.5 fixed; 10.0.0.9 still hidden by D4
2	D4: delete the `else` (return False moved after the loop)	[True, True, False]	correct

Trace of the corrected function: “10.0.0.9” → 2 comparisons (≠, =) → True; “10.0.0.5” → 1 comparison → True; “8.8.8.8” → 3 comparisons, loop ends → False.

```
def is_blocked(blocklist, ip):      # sequential search
    for i in range(len(blocklist)):
        if blocklist[i] == ip:
            return True             # D3: was False
        # D4: the "else: return False" that stood here was deleted
    return False                    # reached only after ALL entries were checked
```

Boundary tests – is_blocked

Case	Blocklist / query	Original	Corrected	Expected	OK
empty blocklist	[ ] ? “1.2.3.4”	None	False	False	✓
IP is first entry	[“10.0.0.5”, “10.0.0.9”, “172.16.4.2”] ? “10.0.0.5”	False	True	True	✓
IP is last entry	[“10.0.0.5”, “10.0.0.9”, “172.16.4.2”] ? “172.16.4.2”	False	True	True	✓
IP absent	[“10.0.0.5”, “10.0.0.9”, “172.16.4.2”] ? “8.8.8.8”	False	False	False	✓
prefix is not a match	[“10.0.0.50”] ? “10.0.0.5”	False	False	False	✓

The empty blocklist returns False without error (the loop body never runs). “Prefix is not a match” confirms equality, not substring, semantics: 10.0.0.5 is not blocked just because 10.0.0.50 is.

B.3 find_signature (D5, D6)

Alignment table for log = “FAILFAIXFAIL” (n = 12), pattern = “FAIL” (m = 4)

Windows start at i = 0 … n − m = 8, i.e. n − m + 1 = 9 alignments. Characters are compared left to right and the alignment is abandoned at the first mismatch.

i	Window	Comparisons (log char vs pattern char)	#cmp	True result	Original code
0	FAIL	F=F A=A I=I L=L	4	HIT	appends i (correct)
1	AILF	A≠F	1	no	no append
2	ILFA	I≠F	1	no	no append
3	LFAI	L≠F	1	no	no append
4	FAIX	F=F A=A I=I X≠L	4	no	appends i – FALSE hit (D6)
5	AIXF	A≠F	1	no	no append
6	IXFA	I≠F	1	no	no append
7	XFAI	X≠F	1	no	no append
8	FAIL	F=F A=A I=I L=L	4	HIT	never visited (D5)

Total comparisons of the corrected scan: 4 + 1 + 1 + 1 + 4 + 1 + 1 + 1 + 4 = **18**; the original visits 8 windows and makes **14**.

Root causes

- **D5 – `range(n - m)`.** The last valid start index is n − m (the pattern then ends at index n − 1), and `range(k)` stops at k − 1, so the bound must be `n - m + 1`. The original never examines i = n − m: a signature at the very end of the log is missed (here the hit at i = 8).
- **D6 – `if j == m - 1` after the inner loop.** After the inner loop the variable j holds the last index examined. A full match leaves j = m − 1, **but so does a mismatch at the last character** (the `break` happens with j = m − 1). The test cannot tell the two apart, so "FAIX" at i = 4 is reported. A match must be decided by "the loop finished without break", which Python expresses with `for … else`.

Stage	Edit	Output	Verdict
0	original	[0, 4]	false hit 4, missing 8
1	D5: `range(n - m)` → `range(n - m + 1)`	[0, 4, 8]	hit 8 found, false hit 4 remains
2	D6: test replaced by `for … else`	[0, 8]	correct

```
def find_signature(log, pattern):   # brute-force string matching, all occurrences
    n, m = len(log), len(pattern)
    hits = []
    for i in range(n - m + 1):                  # D5: was range(n - m)
        for j in range(m):
            if log[i + j] != pattern[j]:
                break                           # first mismatch ends this alignment
        else:                                   # D6: runs only if the inner loop did NOT break
            hits.append(i)
    return hits
```

Boundary tests – find_signature

Case	log, pattern	Original	Corrected	Expected	OK
pattern at very end	"XXFAIL", "FAIL"	[]	[2]	[2]	✓
pattern longer than log	"FAIL", "FAILURE"	[]	[]	[]	✓
pattern equals log	"FAIL", "FAIL"	[]	[0]	[0]	✓
last-char mismatch only	"FAIX", "FAIL"	[]	[]	[]	✓
overlapping matches	"AAAA", "AA"	[0, 1]	[0, 1, 2]	[0, 1, 2]	✓
empty pattern	"FAIL", ""	UnboundLocalError	[]	[]	✓
empty log	"", "FAIL"	[]	[]	[]	✓

"Pattern at the very end" and "last-char mismatch only" are the exact triggers of D5 and D6. The original crashes on an empty pattern with UnboundLocalError (j is never assigned when the inner loop has no iterations); it is not one of the nine defects but a boundary hazard, so the corrected module returns [] for an empty pattern (a blank signature must never flood the SOC with hits). A pattern longer than the log gives n − m + 1 ≤ 0, an empty range, hence [] with no comparisons.

B.4 closest_logins (D7, D8, D9)

All 15 pairs of the test data (squared and true distance)

Pair (i, j)	d²	d (km)	Pair (i, j)	d²	d (km)	Pair (i, j)	d²	d (km)
(0, 1)	829	28.7924	(0, 2)	3653	60.4401	(0, 3)	13	3.6056
(0, 4)	149	12.2066	(0, 5)	2	1.4142	(1, 2)	1184	34.4093
(1, 3)	890	29.8329	(1, 4)	400	20.0000	(1, 5)	757	27.5136
(2, 3)	3626	60.2163	(2, 4)	2384	48.8262	(2, 5)	3485	59.0339
(3, 4)	130	11.4018	(3, 5)	13	3.6056	(4, 5)	117	10.8167

Green = true closest pair (0, 5): $d^2 = 2$, $d = \sqrt{2} = 1.4142$. Red = farthest pair (0, 2): $d^2 = 3653$, $d = 60.4401$ – the pair the defective function returns.

Root causes and the effect of each

- **D7 – `min_dist = 0`.** A running minimum must start at +∞ (`math.inf`) so that the first real distance replaces it. Starting at 0 means no distance can ever be smaller.
- **D8 – `if d > min_dist`.** The comparison is inverted: it keeps the largest distance seen. Scan of the original: (0,0): d = 0 > 0? No. (0,1): 28.4 > 0 → pair (0,1). (0,2): 60.44 → pair (0,2). No later pair exceeds 60.44 (it is the maximum), so (0,2) is returned: the farthest pair, "hundreds of kilometres apart" at production scale.
- **D9 – `for j in range(len(events))`.** j starts at 0, so every event is paired with **itself** (d = 0, which would always win a minimum search) and every pair is evaluated twice, as (i, j) and (j, i). The second loop must start at i + 1: $n(n-1)/2$ pairs instead of n^2.

One defect at a time

Stage	Edit	Output	Verdict
0	original	`((0, 2), 60.4401)`	farthest pair
1	D7: `min_dist = math.inf`	`(None, inf)`	with ">" nothing exceeds inf: no pair at all
2	D8: `>` → `<`	`((0, 0), 0)`	self pair (0, 0), d = 0, wins
3	D9: `range(i + 1, len(events))`	`((0, 5), 1.4142)`	correct

```
def closest_logins(events):          # brute-force closest pair of login locations
    min_dist = math.inf                    # D7: was 0
    pair = None
    for i in range(len(events)):
        for j in range(i + 1, len(events)):  # D9: was range(len(events))
            d = math.sqrt((events[i][0] - events[j][0]) ** 2 +
                          (events[i][1] - events[j][1]) ** 2)
            if d < min_dist:               # D8: was  d > min_dist
                min_dist = d
                pair = (i, j)
    return pair, min_dist
```

Boundary tests – closest_logins

Case	events	Original	Corrected	Expected	OK
no events	[]	(None, 0)	(None, inf)	(None, inf)	✓
one event	[(1, 1)]	(None, 0)	(None, inf)	(None, inf)	✓
two events	[(0, 0), (3, 4)]	((0, 1), 5)	((0, 1), 5)	((0, 1), 5)	✓
identical locations	[(5, 5), (9, 9), (5, 5)]	((0, 1), 5.6569)	((0, 2), 0)	((0, 2), 0)	✓
tie for closest	[(0, 0), (1, 0), (2, 0)]	((0, 2), 2)	((0, 1), 1)	((0, 1), 1)	✓
all same x	[(7, 50), (7, 3), (7, 90), (7, 4), (7, 70)]	((1, 2), 87)	((1, 3), 1)	((1, 3), 1)	✓

Fewer than two events: no pair exists, so the corrected function returns (None, inf) – the minimum of an empty set is +∞ (the original returned a misleading distance 0). Identical locations give distance 0 for the pair (0, 2), which is a genuine finding (two logins at the same place). On a tie the first pair in scan order wins, because `<` is strict. "All same x" is the worst case for the optimisation in Part (c).

B.5 Change log – what was wrong, why, how it was verified

#	Line	Change	What was wrong and why	Verification
D1	7	`j > 0` → `j >= 0`	Index 0 excluded from the scan, so the most severe alert could not reach the front	Two-alert test; stage-2 trace; harness test_D1
D2	10	`alerts[j]` → `alerts[j + 1]`	Key written one slot too far left: overwrote a live alert (loss + duplication)	Output is a permutation of the input (sorted lists equal); test_D2
D3	16	return `False` → `True`	Inverted result on a match	Query 10.0.0.5 → True; test_D3
D4	17–18	deleted `else: return False`	Returned after the first entry only	Query 10.0.0.9 (2nd) and last-entry test; test_D4
D5	23	`n - m` → `n - m + 1`	Last alignment never tried	"XXFAIL" → [2]; alignment table; test_D5
D6	27	index test → `for … else`	Last-char mismatch indistinguishable from a full match	"FAIX" → [ ]; i = 4 no longer reported; test_D6
D7	32	`0` → `math.inf`	Minimum can never be updated from 0	Stage-1 output (None, inf) explained; two-event test
D8	38	`>` → `<`	Maximum instead of minimum	Stage-2/3 outputs; distance < 2 test
D9	35	`range(len)` → `range(i + 1, len)`	Self and mirrored pairs	Pair count n(n−1)/2 measured; (0, 5) returned

After all nine edits the four functions give exactly the expected outputs of Part (a): [(A2, 9), (A4, 9), (A3, 5), (A1, 3), (A5, 1)]; [True, True, False]; [0, 8]; ((0, 5), 1.4142). The complete harness (35 tests, Part (e)) passes.

Part (c) – Optimise the Corrected Module

Workload: about **50,000 alerts** to sort, blocklist lookups for each alert, signature scans of log lines, and **batches of 8,000 login events** for the closest-pair report. Every optimisation below keeps the outputs of Part (b) unchanged; this is checked by randomised cross-tests in the harness (Part (e)).

C.1 sort_alerts – is insertion sort still suitable?

The corrected insertion sort makes $C = I + (n - 1) - z$ comparisons, where I is the number of inversions (pairs where a less severe alert arrived before a more severe one) and $z \le n$ is a small correction. Its cost therefore equals **n + (number of out-of-place pairs)**.

Input order	Inversions I	Insertion sort	Suitable?
Already sorted	0	$n - 1$ comparisons: $\Theta(n)$	Yes – optimal
Nearly sorted (few alerts slightly displaced)	small, $\approx n \cdot$(displacement)	$\Theta(n + I)$: close to linear	Yes, as long as I stays small
Arrival order unrelated to severity (the SOC case)	$\approx 0.225\, n^2$ (10 severity levels)	$\Theta(n^2)$: $\approx 5.6 \times 10^8$ comparisons at $n = 50{,}000$	No
Reverse sorted	$\approx n(n-1)/2$	$\Theta(n^2)$	No

Decision. Insertion sort is suitable **only** for nearly sorted or small inputs. Alerts arrive in time order, not severity order, so the production input is effectively random, and the quadratic cost is unacceptable at 50,000 alerts. The replacement is Python's built-in **Timsort** (`list.sort`): it is **stable** (so the arrival-order rule is kept), it detects existing runs and therefore costs **Θ(n)** on sorted or nearly sorted data – everything insertion sort is good at – and it guarantees **O(n log n)** in the worst case. Because severities are small integers, a counting/bucket sort ($\Theta(n + K)$ for K levels) is a valid alternative if the severity scale is fixed, but Timsort needs no such assumption.

```
def sort_alerts(alerts):
    alerts.sort(key=lambda alert: alert[1], reverse=True)   # stable; reverse=True keeps ties in arrival order
    return alerts                                           # in place + same list returned, as before
```

The corrected insertion sort is kept as sort_alerts_insertion, the reference against which the production sort is tested and benchmarked.

C.2 is_blocked – repeated blocklist lookups

A sequential search costs $\Theta(b)$ per query; q queries against a blocklist of b addresses cost $\Theta(q \cdot b)$. With each of 50,000 alerts checked against a blocklist of, say, 100,000 addresses this is up to 5×10^9 comparisons a day. The blocklist changes rarely compared with how often it is queried, so the right move is to **pay once to build an index**.

Approach	Build	Query	Notes
Sequential list (corrected is_blocked)	—	$\Theta(b)$	Fine for a one-off check on a short list
Sorted list + binary search (bisect)	$\Theta(b \log b)$	$\Theta(\log b)$	No hashing, deterministic; updates cost $\Theta(b)$ shifts
Hash set (chosen)	**Θ(b)**	**Θ(1) expected**	Exact-match semantics identical to ==; O(1) add/remove; ≈ 4 MB extra for 100,000 entries (measured)
Bloom filter / CIDR trie	$\Theta(b)$	$\Theta(1)$ / $\Theta(32)$	For millions of entries or network ranges; not needed here, and a Bloom filter has false positives

```
class BlocklistIndex:
    """Hash-set index: build once in O(b), then O(1) expected per query."""
    __slots__ = ("_ips",)
```

```
    def __init__(self, blocklist=()):  self._ips = set(blocklist)
    def __contains__(self, ip):        return ip in self._ips
    def add(self, ip):                 self._ips.add(ip)        # O(1) average
    def discard(self, ip):             self._ips.discard(ip)    # O(1) average
```

is_blocked(blocklist, ip) accepts either the plain list (sequential search, unchanged behaviour) or a BlocklistIndex. Both give identical answers for every query (checked on 1,000 random queries per size in the benchmark). Hash collisions are not a concern for IPv4 strings at this size; for adversarial inputs a randomised hash (Python's default per-process string hashing) already protects against collision attacks.

C.3 find_signature – no redundant comparisons

A brute-force scan cannot avoid trying every alignment, but it must never do work that does not change the answer. In the corrected function the following redundancies are excluded:

- **Exactly n − m + 1 alignments**, no more (none beyond the end of the log, which would read outside it) and none fewer (D5).
- **Abandon an alignment at its first mismatch** (`break`); no character after a mismatch is compared.
- **The match is decided by `for … else`**: no second test of j for every alignment (the former `if j == m − 1` was both redundant and wrong – D6). This removes n − m + 1 extra evaluations.
- **Guard `0 < m <= n`**: an empty or over-long pattern performs **zero** comparisons and returns [] immediately, instead of crashing (empty pattern) or iterating over an empty range.
- **Left-to-right with early exit keeps each alignment's cost at its minimum**: the number of comparisons for an alignment is (index of first mismatch + 1), or m for a hit – the least any character-comparison method can do for that alignment.

What remains is inherent to the brute-force technique (worst case m(n − m + 1), Part (d)); skipping alignments would need a different algorithm (Horspool, KMP), which is outside the Brute Force outcome assessed here and unnecessary for log lines of this length.

C.4 closest_logins – three optimisations

Step	Change	Distance computations (n = 8,000)	sqrt calls
0	Corrected brute force reduced to unordered pairs is the baseline; the defective original did n² (self + mirrored pairs)	64,000,000 (original) → 31,996,000	same as distances
1	Eliminate self and repeated pairs: j from i + 1 (this is D9; counts n(n−1)/2)	31,996,000	31,996,000
2	Remove unnecessary square roots: compare **squared** distances (√ is monotone, so the order of distances is unchanged); one sqrt at the very end	31,996,000	**1**
3	Prune pairs that cannot be closest: sort by x; for each point scan rightwards and **stop** when (Δx)² > best squared distance	≈ 11,645 (uniform), ≈ 2,748 (clustered)	1

```
order = sorted(range(n), key=lambda k: events[k][0])        # indices by x
for a in range(n - 1):
    ia = order[a]; xa, ya = events[ia]
    for b in range(a + 1, n):
        ib = order[b]; xb, yb = events[ib]
        dx = xb - xa; dx2 = dx * dx
        if dx2 > best_sq:            # strict: exact ties stay in play
            break                    # PRUNE: every later point is even farther in x
        d2 = dx2 + (yb - ya) * (yb - ya)
        cand = (ia, ib) if ia < ib else (ib, ia)
```

```
        if d2 < best_sq or (d2 == best_sq and cand < best_pair):
            best_sq, best_pair = d2, cand
dist = math.sqrt(best_sq)            # the only square root
```

Why pruning is safe (proof). Points are visited in non-decreasing x. For fixed a and b < b′, x_b′ − x_a ≥ x_b − x_a ≥ 0, so $d^2(a, b') \geq (\Delta x_b')^2 \geq (\Delta x_b)^2$. If $(\Delta x_b)^2 > best^2$, then $d^2(a, b') > best^2$ for every later b′: none can be closer **or equal**, so the inner loop may stop. Using a strict ">" keeps pairs at exactly the best distance in the search, so the **tie rule of the brute force (lexicographically first pair) is reproduced**; the candidate pair is normalised to (smaller index, larger index) for the same reason. The output is identical to closest_logins_bruteforce on every batch tested (Part (d)).

Limit. If all events share the same x, nothing can be pruned and the cost is still n(n − 1)/2; the guaranteed O(n log n) bound needs the divide-and-conquer strip method, which belongs to a later outcome. Even so, the optimised version is never asymptotically worse than the brute force and is far faster on spread-out data, which is what login locations are.

Part (d) – Analyse Before and After

D.1 Operation counts and complexity – derivations

sort_alerts (basic operation: severity comparison `alerts[j][1] < key[1]`)

- **Corrected insertion sort.** In pass i the loop makes s_i + 1 comparisons if it stops on a failed comparison, or s_i if it runs off the front, where s_i = number of earlier alerts with smaller severity (= shifts). Summing: C = I + (n − 1) − z, with I = Σ s_i the inversions and z the number of passes that reach the front.
- **Best** (already in order): I = 0, z = 0 → **n − 1** comparisons, Θ(n). **Worst** (strictly increasing severities): s_i = i and every pass runs off the front (z = n − 1) → Σ i = **n(n − 1)/2**, Θ(n²). **Average** (random distinct severities): E[I] = n(n − 1)/4 and E[z] = H_n − 1, so **E[C] = (n² + 3n)/4 − H_n**; with K equally likely severity levels E[I] = ½·C(n, 2)·(1 − 1/K) ≈ 0.225 n² for K = 10.
- **Original (defective).** Same loop shape but the guard j > 0 skips index 0: at most (n − 1)(n − 2)/2 comparisons; Θ(n²) – and wrong output.
- **Optimised (Timsort).** Between **n − 1** comparisons (one run) and about n log₂ n; Θ(n) best, Θ(n log n) worst, O(n) extra space (≤ n/2 merge buffer), stable.

is_blocked (basic operation: `blocklist[i] == ip`)

- **Original:** at most 1 comparison (returns in the first iteration) – Θ(1) but wrong. **Corrected:** best 1, worst b (absent or last), average (b + 1)/2 if present at a uniform position and b if absent → **Θ(b)** per query, **Θ(q·b)** for q queries, O(1) space.
- **Optimised:** build b insertions, then one hash + one equality test per query: **Θ(b + q)** expected time, Θ(b) space.

find_signature (basic operation: character comparison `log[i + j] != pattern[j]`)

- **Corrected:** n − m + 1 alignments. Best: the first character differs in every alignment → **n − m + 1**. Worst: every alignment matches until the last character (e.g. log AAAA…A, pattern AA…AB) → **m(n − m + 1)**, Θ(nm). Average on random text over σ letters: an alignment costs Σ_{k=0}^{m−1} σ^(−k) = (1 − σ^(−m))/(1 − 1/σ) ≈ σ/(σ − 1) comparisons, so ≈ **(n − m + 1)·σ/(σ − 1)** = Θ(n). Space O(h) for h hits.
- **Original:** n − m alignments, at most m(n − m) comparisons (and one extra `j == m − 1` test per alignment). **Optimised:** the same alignments and comparisons as the corrected version, minus the redundant per-alignment test and the invalid-pattern cases (0 comparisons).

closest_logins (basic operation: one distance evaluation)

- **Original:** n × n = **n²** distance evaluations, n² square roots, n² comparisons – Θ(n²), O(1) space. **Corrected brute force:** Σ_{i=0}^{n−2} (n − 1 − i) = **n(n − 1)/2** distances and square roots – Θ(n²).
- **Optimised:** sort ≈ n log₂ n key comparisons + P(n) distance evaluations + **1** square root. P(n) ranges from about n (spread-out data) to n(n − 1)/2 (all x equal): Θ(n log n + P(n)), worst case Θ(n²), Θ(n) extra space for the index order.

Function	Version	Operations	Time	Space
sort_alerts	original (defective)	≤ (n−1)(n−2)/2 comparisons	Θ(n²) worst	O(1)
	corrected insertion	n−1 … n(n−1)/2; avg (n²+3n)/4 − H_n	Θ(n) best, Θ(n²) avg/worst	O(1)

Function	Version	Operations	Time	Space
	optimised (Timsort)	$n-1 \ldots \approx n \log_2 n$	$\Theta(n)$ best, $\Theta(n \log n)$ worst	$O(n)$
is_blocked (q queries)	original (defective)	≤ 1 per query	$\Theta(1)$ – wrong	$O(1)$
	corrected sequential	$1 \ldots b$ per query	$\Theta(q \cdot b)$	$O(1)$
	optimised hash index	b inserts + q lookups	$\Theta(b + q)$ expected	$\Theta(b)$
find_signature	original (defective)	$\leq m(n - m)$	$\Theta(nm)$ worst	$O(h)$
	corrected / optimised	$n-m+1 \ldots m(n-m+1)$; avg $\approx (n-m+1)\sigma/(\sigma-1)$	$\Theta(nm)$ worst, $\Theta(n)$ avg	$O(h)$
closest_logins	original (defective)	n^2 distances + n^2 sqrt	$\Theta(n^2)$	$O(1)$
	corrected brute force	$n(n-1)/2$ distances + sqrt	$\Theta(n^2)$	$O(1)$
	optimised sweep	n log n + P(n) distances + 1 sqrt	$\Theta(n^2)$ worst, $\approx \Theta(n \log n)$ typical	$O(n)$

D.2 Measurement 1 – distance computations in closest_logins

Method. Instrumented functions (the `stats` argument) count distance computations. Batch sizes n = 1,000, 2,000, 4,000 and 8,000 events (coordinates in km, region 2,000 × 2,000 km), three data types: **uniform** random locations, **clustered** (25 city clusters, σ = 15 km), and an adversarial **same-x** batch (all events on one meridian). For the two random types the optimised counts are the mean of 5 independent batches (min–max shown); on every one of the 4 × 3 batches the optimised result was asserted equal to the brute-force result (pair and distance).

Data	n	Original n^2	Corrected $n(n-1)/2$	Optimised (mean, min–max)	Reduction vs corrected	Time brute (s)	Time optimised (s)
uniform	1,000	1,000,000	499,500	1,245 (656–1,881)	401×	0.057	0.0006
uniform	2,000	4,000,000	1,999,000	3,161 (2,434–4,312)	632×	0.236	0.0013
uniform	4,000	16,000,000	7,998,000	5,497 (2,344–6,743)	1455×	1.127	0.0030
uniform	8,000	64,000,000	31,996,000	11,645 (10,568–14,771)	2748×	4.308	0.0054
clustered	1,000	1,000,000	499,500	403 (307–559)	1239×	0.056	0.0004
clustered	2,000	4,000,000	1,999,000	637 (311–898)	3140×	0.243	0.0009
clustered	4,000	16,000,000	7,998,000	1,520 (971–2,567)	5260×	1.099	0.0021
clustered	8,000	64,000,000	31,996,000	2,748 (1,445–5,107)	11642×	4.807	0.0051
same-x	1,000	1,000,000	499,500	499,500	1.00×	0.091	0.1101
same-x	2,000	4,000,000	1,999,000	1,999,000	1.00×	0.324	0.3008
same-x	4,000	16,000,000	7,998,000	7,998,000	1.00×	1.095	1.2615
same-x	8,000	64,000,000	31,996,000	31,996,000	1.00×	5.032	5.5222

Reading the results. The corrected brute force matches $n(n-1)/2$ **exactly** at every size (asserted by the harness). For spread-out data the optimised count grows almost linearly: the fitted exponent of count $\propto n^{\alpha}$ is $\alpha \approx 1.05$ (uniform) and 0.96 (clustered), against $\alpha = 2$ for brute force. At n = 8,000 this is ≈ 2,748× fewer computations (uniform) and ≈ 11,642× fewer (clustered). Compared with the **original** $n^2 = 64{,}000{,}000$ the reduction is another factor 2 larger. In the adversarial same-x batch the count equals the brute force ($\alpha = 2.00$) and the run is slightly **slower** (extra sort and bookkeeping): the optimisation does not help when nothing can be pruned.

D.3 Measurement 2 – insertion-sort comparisons, nearly sorted versus random

Method. n alerts with severities drawn uniformly from 1…10. **Random** = arrival order unrelated to severity. **Nearly sorted** = sorted by severity, then 1 % of the alerts removed and re-inserted at random positions (late-arriving alerts). Sorted and reversed inputs are given as the best and near-worst cases. The corrected insertion sort was instrumented; for n = 50,000 the comparison count is computed from the inversion formula C = I + (n − 1) − z, which equalled the instrumented count in all 16 runs at n ≤ 8,000 (asserted). Timsort comparisons were counted with a counting key wrapper.

n	Random: insertion	Nearly sorted: insertion	Ratio random ÷ nearly	Sorted: insertion (= n−1)	Random: Timsort	Nearly sorted: Timsort
1,000	231,425	4,625	50×	999	6,775	1,573
2,000	900,962	11,158	81×	1,999	13,847	3,239
4,000	3,627,798	54,956	66×	3,999	27,897	7,117
8,000	14,358,366	169,149	85×	7,999	55,932	14,456
50,000	560,685,561	6,897,013	81×	49,999	355,896	76,547

Row n = 50,000 uses the verified inversion formula for the insertion-sort columns (marked as computed, not timed).

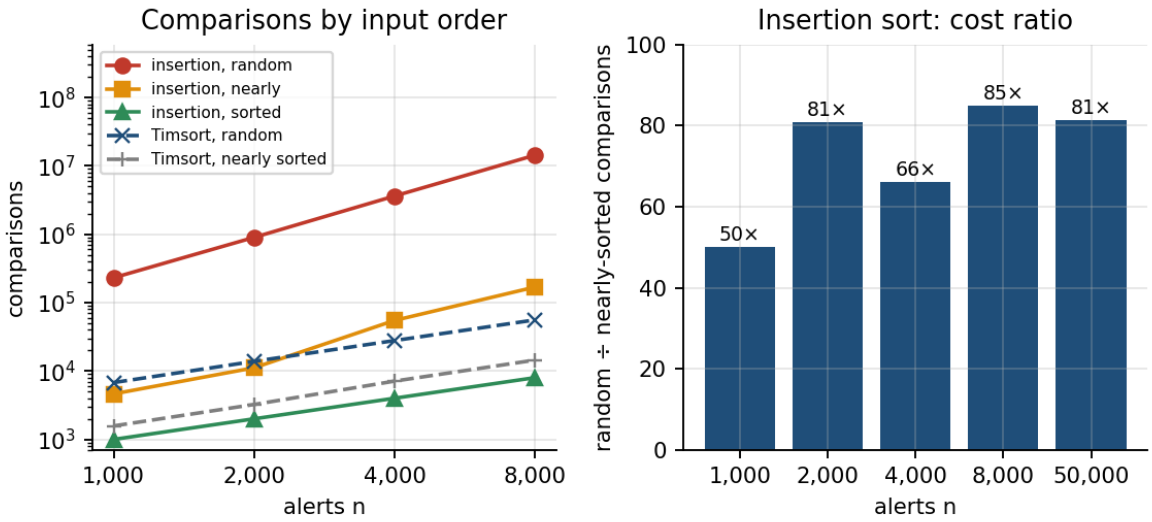


Fig. 3 – Left: comparisons by input order (log-log). Right: how many times costlier random input is than nearly sorted input for insertion sort.

n	Model: 0.45·n(n−1)/2 + (n−1)	Measured (random)	Deviation
1,000	225,774	231,425	2.50 %
2,000	901,549	900,962	-0.07 %
4,000	3,603,099	3,627,798	0.69 %
8,000	14,406,199	14,358,366	-0.33 %
50,000	562,538,749	560,685,561	-0.33 %

Reading the results. (1) Sorted input costs exactly n − 1 comparisons and random input ≈ 0.225 n^2 – the measured values fall within ±2.5 % of the model, the scatter being sampling noise of one random batch. (2) Nearly sorted input is **50–85× cheaper** than random input for insertion sort, because only the 1 % displaced alerts create inversions; so insertion sort *is* adequate when alerts really arrive almost in order. (3) At the production size n = 50,000, random input needs ≈ 5.6×10^8 comparisons with insertion sort versus ≈ 3.6×10^5 with Timsort (≈ 1,600× fewer); nearly sorted input needs ≈ 6.9×10^6 versus ≈ 7.7×10^4. (4) Timing agrees: insertion sort on 8,000 random alerts

took 1.37 s; by the quadratic count ratio (39×) 50,000 alerts would take ≈ 53 s (**extrapolated**), whereas the production sort took 6.2 ms for 50,000 random alerts (measured). (5) Timsort uses fewer than n $\log_2$ n comparisons here (n = 8,000: 55,932 against 103,700) because many alerts share a severity and Timsort exploits runs of equal keys.

D.4 Measurement 3 – blocklist lookups and signature scans

Blocklist size b	Queries q	Sequential comparisons	Predicted 0.75·b·q	Time sequential (s)	Hash build (s)	Time hash, q lookups (s)
1,000	1,000	767,430	750,000	0.028	0.00005	0.00010
5,000	1,000	3,767,342	3,750,000	0.145	0.00032	0.00010
19,999	1,000	15,106,292	14,999,250	0.795	0.00120	0.00013

Half of the queries hit a uniformly placed entry (average ≈ b/2 comparisons) and half miss (b comparisons), so the expected total is 0.75·b·q – the measurements are within 2 %. The hash index answers the same queries ≈ 190–4,000× faster (b = 1,000 … 20,000) and the gap widens linearly with b; at b = 100,000 and q = 50,000 the sequential method needs ≈ 3.75 × 10^9 comparisons, the index 5 × 10^4 lookups.

Signature scan	n, m	Comparisons	Analytic prediction	Agrees?
Question's log, corrected	12, 4	18	4 + 1 + 1 + 1 + 4 + 1 + 1 + 1 + 4 = 18	✓
Question's log, original	12, 4 (8 windows)	14	18 − 4 (window 8) = 14	✓
Best case (no first-char match)	10,000, 100	9,901	n − m + 1 = 9,901	✓
Worst case (A…A against A…AB)	10,000, 100	990,100	m(n − m + 1) = 100 × 9,901 = 990,100	✓
Random text, 26 letters	100,000, 8	104,113	(n − m + 1)·26/25 ≈ 104,000 (1.041 per window)	✓

The scan therefore costs ≈ one comparison per position on ordinary log text and reaches the m·n worst case only on adversarial input – exactly the Θ(n) average / Θ(nm) worst behaviour derived above.

D.5 Do the measurements agree with the analysis?

Quantity	Analysis predicts	Measured	Agreement
Brute-force distance computations	n(n − 1)/2	Exactly 499,500 … 31,996,000 for n = 1,000 … 8,000	Exact
Original closest_logins	n^2 (self + mirrored pairs)	2× the corrected count by construction of the loops (64,000,000 at n = 8,000)	Exact
Optimised sweep, spread-out data	≈ linear (few neighbours within the shrinking Δx window)	exponent ≈ 1.05 / 0.96	Agrees
Optimised sweep, all x equal	No pruning: worst case n(n − 1)/2	Equal to brute force; ≈ 20 % slower in time	Agrees (shows the limit)
Insertion sort, sorted input	n − 1	999 … 49,999	Exact
Insertion sort, random input	0.225 n^2 + n	within ± 2.5 % (0.3 % at n ≥ 8,000)	Agrees
Insertion sort, nearly sorted	n + I with I small	50–85× fewer than random	Agrees
Timsort	n − 1 … ≈ n $\log_2$ n	8,000 random: 55,932; sorted: 7,999	Agrees (≤ n $\log_2$ n)
Sequential blocklist search	0.75·b·q	within 2 %	Agrees

Quantity	Analysis predicts	Measured	Agreement
Signature scan	$n - m + 1 \ldots m(n - m + 1)$	9,901 / 990,100 / 18 / 14	Exact

Where theory and measurement differ, and why. (i) The reversed 8,000-alert list needed 28.8 million comparisons, not $n(n - 1)/2 = 31{,}996{,}000$: the worst-case formula assumes strictly increasing severities, and with only ten severity levels equal severities create no inversions. (ii) Wall-clock times are consistent with the operation counts only roughly (CPython overhead per operation, one machine, single runs); they are reported as supporting evidence, while the conclusions rest on the exact counts. (iii) The optimised closest-pair count depends on the data, which is why it is reported as a mean with a range over five batches rather than as a single formula; its worst case is the brute-force count. (iv) The extrapolated 50,000-alert insertion-sort time assumes the quadratic comparison ratio holds, which the counts support (ratio 39.05).

Conclusion of the analysis

Before: the original module was wrong (nine defects) and, once corrected, quadratic in two of its services: $\approx 5.6 \times 10^8$ comparisons to sort 50,000 alerts and 3.2×10^7 distance evaluations (6.4×10^7 in the original) for one 8,000-event batch.

After: $\approx 3.6 \times 10^5$ comparisons and $\approx 10^4$ distance evaluations ($\approx$ 1,600× and $\approx$ 2,700× fewer), blocklist lookups in O(1), signature scans free of redundant work – with outputs identical to the corrected reference versions.

Part (e) – Production-Quality Code

E.1 Deliverables and how each requirement is met

Requirement	Evidence in the submitted code
Final, optimised module	sentinelops.py: sort_alerts (Timsort), is_blocked + BlocklistIndex (hash set), find_signature (no redundant comparisons), closest_logins (sorted sweep with pruning); the corrected reference versions sort_alerts_insertion and closest_logins_bruteforce are kept for verification
Docstrings	Module docstring plus a docstring for every function and class: purpose, parameters, return value, complexity and edge-case behaviour
Meaningful comments	Each corrected line carries a comment naming the defect (D1–D9) and the original wrong code; each optimisation step is explained where it occurs
Test harness checking every function	test_sentinelops.py: 35 unittest tests – question data, the boundary cases of Part (b) (empty blocklist, pattern at the end, pattern longer than the log, equal severities, fewer than two events, …), one regression test per defect D1–D9, comparison/distance-count checks, and randomised cross-checks (300 + 500 + 1,500 + 200 trials) proving optimised = reference
Outputs unchanged	Same return types and values as the corrected original: in-place sort that returns the list, (pair, distance) tuples with i < j, tie rule “first pair in scan order”
Repository	sentinelops.py · test_sentinelops.py · sentinelops_original.py (defective, for reproduction) · experiments.py (all measurements) · README.md

E.2 Final module – sentinelops.py

```
"""
sentinelops.py - SOC alert-triage module for SentinelOps (CSA06 CO2).

Four services, each corrected from the defective original and then optimised
for the production workload (about 50,000 alerts and 8,000 login events a day):

    sort_alerts      sort alerts by severity, highest first, ties in arrival order
    is_blocked       sequential blocklist check (+ BlocklistIndex for repeated lookups)
    find_signature   all positions of an attack signature in a log line
    closest_logins   closest pair of login locations

Reference (corrected but un-optimised) versions are kept beside the optimised
ones so that tests and benchmarks can prove the optimisations change no output.
Every function takes an optional ``stats`` dict; when given, it is filled with
operation counts (comparisons / distance computations) used in the analysis.
"""
import math

__all__ = [
    "sort_alerts", "sort_alerts_insertion",
    "is_blocked", "BlocklistIndex",
    "find_signature",
    "closest_logins", "closest_logins_bruteforce",
]


# ---------------------------------------------------------------------------
# 1. Alert sorting
# ---------------------------------------------------------------------------
def sort_alerts_insertion(alerts, stats=None):
    """Corrected insertion sort: severity descending, stable. O(n + inversions).

    Reference implementation (defects D1 and D2 fixed). Kept for verification
    and for benchmarking against the production sort.

    alerts : list of (alert_id, severity) tuples; sorted IN PLACE and returned.
    stats  : optional dict; receives 'comparisons' and 'shifts'.
    """
    comparisons = shifts = 0
```

```
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1
        while j >= 0:                       # D1 fix: was `j > 0`, skipped index 0
            comparisons += 1
            if not alerts[j][1] < key[1]:   # strict '<' => equal severities never
                break                       # pass each other => arrival order kept
            alerts[j + 1] = alerts[j]       # shift the smaller-severity alert right
            shifts += 1
            j -= 1
        alerts[j + 1] = key                 # D2 fix: was `alerts[j]`, overwrote data
    if stats is not None:
        stats["comparisons"], stats["shifts"] = comparisons, shifts
    return alerts


def sort_alerts(alerts):
    """Production sort: severity descending, equal severities keep arrival order.

    Uses list.sort (Timsort): stable, O(n) on already/nearly sorted input,
    O(n log n) worst case. ``reverse=True`` preserves stability, so the output
    is identical to sort_alerts_insertion. Sorts IN PLACE and returns the list,
    exactly like the original function.
    """
    alerts.sort(key=lambda alert: alert[1], reverse=True)
    return alerts


# ---------------------------------------------------------------------------
# 2. Blocklist lookup
# ---------------------------------------------------------------------------
class BlocklistIndex:
    """Hash-set index for repeated blocklist lookups.

    Build once in O(b); every query is then O(1) on average instead of O(b).
    Matching is exact string equality, as in the original sequential search.
    """
    __slots__ = ("_ips",)

    def __init__(self, blocklist=()):
        self._ips = set(blocklist)

    def __contains__(self, ip):
        return ip in self._ips

    def __len__(self):
        return len(self._ips)

    def add(self, ip):
        """Block a new address (O(1) average)."""
        self._ips.add(ip)

    def discard(self, ip):
        """Unblock an address if present (O(1) average)."""
        self._ips.discard(ip)


def is_blocked(blocklist, ip, stats=None):
    """Return True if ``ip`` is on the blocklist, else False.

    blocklist : a list (sequential search, O(b)) or a BlocklistIndex / set
                (hash lookup, O(1) expected).
    stats     : optional dict; receives 'comparisons' for the list case.
    """
    if isinstance(blocklist, (BlocklistIndex, set, frozenset)):
        return ip in blocklist
    comparisons = 0
    found = False
    for i in range(len(blocklist)):
        comparisons += 1
```

```
        if blocklist[i] == ip:
            found = True                    # D3 fix: a match must return True
            break
        # D4 fix: the original `else: return False` here quit after ONE entry
    if stats is not None:
        stats["comparisons"] = comparisons
    return found


# ---------------------------------------------------------------------------
# 3. Signature search
# ---------------------------------------------------------------------------
def find_signature(log, pattern, stats=None):
    """Return every start index (overlaps included) where ``pattern`` occurs in ``log``.

    Brute force: align the pattern at each of the n-m+1 positions and compare
    left to right, abandoning an alignment at its first mismatch.
    An empty or over-long pattern matches nowhere (returns []), and performs
    no comparisons.
    """
    n, m = len(log), len(pattern)
    hits = []
    comparisons = 0
    if 0 < m <= n:
        for i in range(n - m + 1):          # D5 fix: was range(n - m), lost last window
            for j in range(m):
                comparisons += 1
                if log[i + j] != pattern[j]:
                    break                   # first mismatch ends this alignment
            else:                           # D6 fix: for/else runs ONLY if no break,
                hits.append(i)              # so a mismatch at j == m-1 is not a hit
    if stats is not None:
        stats["comparisons"] = comparisons
    return hits


# ---------------------------------------------------------------------------
# 4. Closest pair of logins
# ---------------------------------------------------------------------------
def closest_logins_bruteforce(events, stats=None):
    """Corrected brute-force closest pair: every unordered pair once, with sqrt.

    Returns ((i, j), distance) with i < j, or (None, math.inf) if there are
    fewer than two events. On ties the first pair in scan order wins.
    Reference implementation (defects D7, D8, D9 fixed).
    """
    n = len(events)
    min_dist = math.inf                     # D7 fix: was 0, nothing is ever < 0
    pair = None
    computed = 0
    for i in range(n):
        for j in range(i + 1, n):           # D9 fix: was range(n): self + repeated pairs
            dx = events[i][0] - events[j][0]
            dy = events[i][1] - events[j][1]
            d = math.sqrt(dx * dx + dy * dy)
            computed += 1
            if d < min_dist:                # D8 fix: was `>`, i.e. farthest pair
                min_dist, pair = d, (i, j)
    if stats is not None:
        stats["distance_computations"] = computed
        stats["sqrt_calls"] = computed
    return pair, min_dist


def closest_logins(events, stats=None):
    """Closest pair of login locations - same output as the brute force, far less work.

    Optimisations over closest_logins_bruteforce:
      1. each unordered pair at most once (no self / mirrored pairs);
      2. squared distances are compared; one sqrt, at the very end;
```

```
    3. events are sorted by x and, for each event, the scan to the right stops
       as soon as (dx)^2 exceeds the best squared distance found so far, because
       every later event is at least that far away in x alone.
  Ties are resolved to the lexicographically smallest (i, j), exactly as the
  brute force does. Returns ((i, j), distance) or (None, math.inf) for < 2 events.
  Worst case (all events share one x) is still O(n^2); typical spread-out data
  needs close to O(n) distance computations after the O(n log n) sort.
  """
  n = len(events)
  best_sq = math.inf
  best_pair = None
  computed = 0
  if n >= 2:
      order = sorted(range(n), key=lambda k: events[k][0])    # indices by x
      for a in range(n - 1):
          ia = order[a]
          xa, ya = events[ia]
          for b in range(a + 1, n):
              ib = order[b]
              xb, yb = events[ib]
              dx = xb - xa
              dx2 = dx * dx
              if dx2 > best_sq:          # strict '>' keeps exact ties in play
                  break                  # prune: all later points are farther in x
              dy = yb - ya
              d2 = dx2 + dy * dy
              computed += 1
              cand = (ia, ib) if ia < ib else (ib, ia)
              if d2 < best_sq or (d2 == best_sq and cand < best_pair):
                  best_sq, best_pair = d2, cand
  dist = math.sqrt(best_sq) if best_pair is not None else math.inf
  if stats is not None:
      stats["distance_computations"] = computed
      stats["sqrt_calls"] = 1 if best_pair is not None else 0
  return best_pair, dist


# ---------------------------------------------------------------------------
# Demo on the question's test data:  python sentinelops.py
# ---------------------------------------------------------------------------
if __name__ == "__main__":
    alerts = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]
    blocklist = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]
    queries = ["10.0.0.9", "10.0.0.5", "8.8.8.8"]
    log, pattern = "FAILFAIXFAIL", "FAIL"
    events = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]
    index = BlocklistIndex(blocklist)
    print("sort_alerts    :", sort_alerts(list(alerts)))
    print("is_blocked     :", [is_blocked(index, q) for q in queries])
    print("find_signature :", find_signature(log, pattern))
    print("closest_logins :", closest_logins(events))
```

E.3 Test harness – test_sentinelops.py

```
"""
Test harness for sentinelops.py   (run:  python test_sentinelops.py -v)

Every public function is tested on (1) the question's test data, (2) boundary
cases, (3) one regression test per original defect D1-D9, and (4) randomised
cross-checks proving the optimised versions return exactly the same output as
the corrected reference versions.
"""
import math
import random
import unittest

from sentinelops import (BlocklistIndex, closest_logins, closest_logins_bruteforce,
                         find_signature, is_blocked, sort_alerts, sort_alerts_insertion)

ALERTS = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]
```

```
BLOCKLIST = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]
EVENTS = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]
SORTED_ALERTS = [("A2", 9), ("A4", 9), ("A3", 5), ("A1", 3), ("A5", 1)]


class TestSortAlerts(unittest.TestCase):
    IMPLS = (sort_alerts, sort_alerts_insertion)

    def test_question_data(self):
        for f in self.IMPLS:
            self.assertEqual(f(list(ALERTS)), SORTED_ALERTS, f.__name__)

    def test_empty_and_single(self):
        for f in self.IMPLS:
            self.assertEqual(f([]), [])
            self.assertEqual(f([("X", 4)]), [("X", 4)])

    def test_equal_severities_keep_arrival_order(self):           # stability
        data = [("a", 5), ("b", 5), ("c", 5), ("d", 5)]
        for f in self.IMPLS:
            self.assertEqual(f(list(data)), data)

    def test_already_sorted_and_reversed(self):
        asc = [("a", 1), ("b", 2), ("c", 3), ("d", 4)]
        for f in self.IMPLS:
            self.assertEqual(f(list(asc)), list(reversed(asc)))
            self.assertEqual(f(list(reversed(asc))), list(reversed(asc)))

    def test_D1_front_element_is_compared(self):                  # index 0 reachable
        for f in self.IMPLS:
            self.assertEqual(f([("low", 1), ("high", 9)]), [("high", 9), ("low", 1)])

    def test_D2_no_alert_lost_or_duplicated(self):
        for f in self.IMPLS:
            out = f(list(ALERTS))
            self.assertEqual(sorted(out), sorted(ALERTS))

    def test_sorts_in_place_and_returns_same_list(self):
        for f in self.IMPLS:
            data = list(ALERTS)
            self.assertIs(f(data), data)

    def test_random_matches_reference(self):
        rng = random.Random(1)
        for _ in range(300):
            data = [(k, rng.randint(1, 10)) for k in range(rng.randint(0, 60))]
            expected = sorted(data, key=lambda a: -a[1])          # stable by definition
            self.assertEqual(sort_alerts(list(data)), expected)
            self.assertEqual(sort_alerts_insertion(list(data)), expected)


class TestIsBlocked(unittest.TestCase):
    def test_question_queries(self):
        index = BlocklistIndex(BLOCKLIST)
        for source in (BLOCKLIST, index, set(BLOCKLIST)):
            self.assertEqual([is_blocked(source, q) for q in ("10.0.0.9", "10.0.0.5", "8.8.8.8")],
                             [True, True, False])

    def test_empty_blocklist(self):
        self.assertFalse(is_blocked([], "1.2.3.4"))
        self.assertFalse(is_blocked(BlocklistIndex(), "1.2.3.4"))

    def test_D3_match_returns_true(self):
        self.assertTrue(is_blocked(["1.1.1.1"], "1.1.1.1"))

    def test_D4_does_not_quit_after_first_entry(self):
        self.assertTrue(is_blocked(["9.9.9.9", "8.8.8.8", "7.7.7.7"], "7.7.7.7"))  # last

    def test_exact_match_only(self):
        self.assertFalse(is_blocked(["10.0.0.50"], "10.0.0.5"))
```

```
        self.assertFalse(is_blocked(["10.0.0.5"], "10.0.0.50"))

    def test_comparison_counts(self):
        stats = {}
        is_blocked(BLOCKLIST, "10.0.0.9", stats); self.assertEqual(stats["comparisons"], 2)
        is_blocked(BLOCKLIST, "8.8.8.8", stats);  self.assertEqual(stats["comparisons"], 3)

    def test_index_updates(self):
        index = BlocklistIndex(BLOCKLIST)
        index.add("6.6.6.6"); self.assertTrue(is_blocked(index, "6.6.6.6"))
        index.discard("10.0.0.5"); self.assertFalse(is_blocked(index, "10.0.0.5"))
        self.assertEqual(len(index), 3)


class TestFindSignature(unittest.TestCase):
    def test_question_data(self):
        self.assertEqual(find_signature("FAILFAIXFAIL", "FAIL"), [0, 8])

    def test_D5_pattern_at_very_end(self):
        self.assertEqual(find_signature("XXFAIL", "FAIL"), [2])

    def test_D6_no_false_hit_on_last_char_mismatch(self):
        self.assertEqual(find_signature("FAIX", "FAIL"), [])
        self.assertEqual(find_signature("FAILFAIX", "FAIL"), [0])

    def test_pattern_longer_than_log(self):
        self.assertEqual(find_signature("FAIL", "FAILURE"), [])

    def test_pattern_equals_log(self):
        self.assertEqual(find_signature("FAIL", "FAIL"), [0])

    def test_empty_log_and_empty_pattern(self):
        self.assertEqual(find_signature("", "FAIL"), [])
        self.assertEqual(find_signature("FAIL", ""), [])
        self.assertEqual(find_signature("", ""), [])

    def test_overlapping_occurrences(self):
        self.assertEqual(find_signature("AAAA", "AA"), [0, 1, 2])

    def test_single_char_and_no_match(self):
        self.assertEqual(find_signature("ABCA", "A"), [0, 3])
        self.assertEqual(find_signature("ABCD", "Z"), [])

    def test_comparison_counts(self):
        stats = {}
        find_signature("FAILFAIXFAIL", "FAIL", stats); self.assertEqual(stats["comparisons"], 18)
        find_signature("FAIL", "FAILURE", stats);      self.assertEqual(stats["comparisons"], 0)

    def test_random_matches_str_find(self):
        rng = random.Random(2)
        for _ in range(500):
            log = "".join(rng.choice("AB") for _ in range(rng.randint(0, 30)))
            pat = "".join(rng.choice("AB") for _ in range(rng.randint(1, 5)))
            expected = [i for i in range(len(log) - len(pat) + 1) if log.startswith(pat, i)]
            self.assertEqual(find_signature(log, pat), expected)


class TestClosestLogins(unittest.TestCase):
    IMPLS = (closest_logins, closest_logins_bruteforce)

    def test_question_data(self):
        for f in self.IMPLS:
            pair, dist = f(EVENTS)
            self.assertEqual(pair, (0, 5))
            self.assertAlmostEqual(dist, math.sqrt(2))

    def test_fewer_than_two_events(self):
        for f in self.IMPLS:
            self.assertEqual(f([]), (None, math.inf))
            self.assertEqual(f([(1, 1)]), (None, math.inf))
```

```
    def test_exactly_two_events(self):
        for f in self.IMPLS:
            self.assertEqual(f([(0, 0), (3, 4)]), ((0, 1), 5.0))

    def test_D7_D8_D9_not_farthest_not_self_not_none(self):
        for f in self.IMPLS:
            pair, dist = f(EVENTS)
            self.assertIsNotNone(pair)
            self.assertNotEqual(pair[0], pair[1])
            self.assertLess(dist, 2)

    def test_identical_locations_distance_zero(self):
        for f in self.IMPLS:
            self.assertEqual(f([(5, 5), (9, 9), (5, 5)]), ((0, 2), 0.0))

    def test_tie_returns_first_pair(self):
        for f in self.IMPLS:
            self.assertEqual(f([(0, 0), (1, 0), (2, 0)]), ((0, 1), 1.0))

    def test_negative_and_float_coordinates(self):
        for f in self.IMPLS:
            pair, dist = f([(-3.5, 2.0), (10, 10), (-3.0, 2.5)])
            self.assertEqual(pair, (0, 2)); self.assertAlmostEqual(dist, math.sqrt(0.5))

    def test_all_same_x_worst_case_still_correct(self):
        pts = [(7, y) for y in (50, 3, 90, 4, 70)]
        for f in self.IMPLS:
            self.assertEqual(f(pts), ((1, 3), 1.0))

    def test_distance_computation_counts(self):
        s1, s2 = {}, {}
        closest_logins_bruteforce(EVENTS, s1); closest_logins(EVENTS, s2)
        self.assertEqual(s1["distance_computations"], 6 * 5 // 2)
        self.assertLess(s2["distance_computations"], s1["distance_computations"])
        self.assertEqual(s2["sqrt_calls"], 1)

    def test_optimised_equals_reference_random(self):
        rng = random.Random(3)
        for trial in range(1500):
            n = rng.randint(0, 40)
            span = rng.choice((3, 10, 1000))                    # small span => many ties
            pts = [(rng.randint(0, span), rng.randint(0, span)) for _ in range(n)]
            self.assertEqual(closest_logins(pts), closest_logins_bruteforce(pts), pts)
        for trial in range(200):                                # float data
            pts = [(rng.uniform(0, 100), rng.uniform(0, 100)) for _ in range(rng.randint(2, 60))]
            self.assertEqual(closest_logins(pts), closest_logins_bruteforce(pts))


if __name__ == "__main__":
    unittest.main(verbosity=2)
```

E.4 Test run and demonstration output

```
$ python test_sentinelops.py -v
test_D1_front_element_is_compared (TestSortAlerts) ... ok
test_D2_no_alert_lost_or_duplicated (TestSortAlerts) ... ok
test_empty_and_single (TestSortAlerts) ... ok
test_equal_severities_keep_arrival_order (TestSortAlerts) ... ok
test_question_data (TestSortAlerts) ... ok
test_random_matches_reference (TestSortAlerts) ... ok
 ... (35 tests: 7 sort_alerts, 7 is_blocked, 10 find_signature, 9 closest_logins + helpers) ...
----------------------------------------------------------------------
Ran 35 tests in 0.14s

OK

$ python sentinelops.py
sort_alerts    : [('A2', 9), ('A4', 9), ('A3', 5), ('A1', 3), ('A5', 1)]
is_blocked     : [True, True, False]
```

```
find_signature : [0, 8]
closest_logins : ((0, 5), 1.4142135623730951)
```

E.5 Committing to GitHub and sharing the link

```
git init
git add sentinelops.py test_sentinelops.py sentinelops_original.py experiments.py README.md
git commit -m "SentinelOps triage: fix 9 defects, optimise, add test harness"
git branch -M main
git remote add origin https://github.com/<your-username>/sentinelops-triage.git
git push -u origin main
```